Programowanie obiektowe i inżynieria oprogramowania

Lab 03:

- zmienna bardzo globalna,
- poszukiwanie elementu w wektorze/tablicy.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2024

Zmienna bardzo globalna

Zmienna bardzo globalna

W pliku nagłówkowym dodajmy **deklaracje** (informacje o istnieniu) zmiennej (wektora przechowującego obiekty **klasy/typu** *TSubstance*). Dodanie *extern* oznacza, że zmienna gdzieś zostanie **zdefiniowana** (utworzona). Gdzie dokładnie? W tej chwili nie wiadomo.

Dołączając plik nagłówkowy do innego pliku źródłowego informujemy funkcje z pliku źródłowego, że taka zmienna (ze słówkiem extern) zostanie gdzieś utworzona i możemy z niej korzystać w zakresie tego pliku. W ten sposób uzyskujemy zmienną globalną, z której można korzystać w wielu plikach.

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1  #pragma once
2  #include <iostream>
3  #include <vector>
4
5
6  class TSubstance
7  {
8      std::string name;
9      double      ro;       // [kg/m3]
10
11 public:
12     TSubstance(std::string name, double ro);
13
14     std::string get_name();
15     double      get_ro();
16 };
17
18 extern std::vector<TSubstance> substancje;
19
```

Deklaracja -> informacja o istnieniu
Definicja -> powołanie do życia

---

*„(wektora przechowującego obiekty **klasy/typu** TSubstance)”*

Autor C++ położył mocny nacisk na interpretowaniu klas jako zmiennych typu zdefiniowanego przez użytkownika.

Tak jak mamy typy wbudowane (zmienne typu *int*, *double* itd.), tak np. wektor jest zmienną typu zdefiniowanego przez użytkownika C++. Podobnie można interpretować własne klasy. Obiekty naszych klas to **zmienne** typu klasy.

Zmienna bardzo globalna

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```cpp
#include "TSubstance.h"
using namespace std;

TSubstance::TSubstance(string name, double ro)
{
    this->name = name;
    this->ro = ro;
}

string TSubstance::get_name()
{
    return name;
}

double TSubstance::get_ro()
{
    return ro;
}


vector<TSubstance> substancje =
{
    TSubstance{"woda", 997.8},
    TSubstance{"mleko", 1030.0},
    TSubstance{"oliwa", 920.0},
    TSubstance{"krew", 1050.0}
};
```

Przerzucamy menu substancji (wektor *substancje*) do pliku źródłowego klasy *TSubstance*.

W ten sposób wszystkie istotne sprawy związane z substancjami znajdują się w jednym miejscu.

W pliku źródłowym z funkcją *main* możemy skupić się na istotnych kwestiach związanych z przeznaczeniem programu. Kod staje się bardziej przejrzysty.

Konsola debugowania programu Microsoft Visual Studio

```
Hello World!
0. woda; volume: 100 ml; mass: 99.78 g
1. mleko; volume: 50 ml; mass: 51.5 g
Volume coe: 66.6667 - 33.3333 -
Mass coe: 65.9572 - 34.0428 -
```

Dodawanie płynu po nazwie

Dodanie płynu po nazwie

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1   #pragma once
2   #include <iostream>
3   #include <vector>
4
5   #include "TSubstance.h"
6
7
8   class TCup
9   {
10      std::vector<TSubstance> substances;
11      std::vector<double> volumes;
12
13  public:
14      void add(TSubstance substance, double volume_in_ml);
15      void add(std::string name, double volume_in_ml);
16      void show();
17
18  private:
19      int get_substance_id(std::string name);
20  };
```

Linia 15

Dodajemy nową funkcję umożliwiającą dodawanie substancji do kubka. Tym razem dajemy możliwość dodania cieczy poprzez podanie jej nazwy – występującej w menu, zmiennej globalnej.

Wychodzimy z założenia, że użytkownikowi łatwiej jest wpisać nazwę płynu niż pamiętać jej pozycje w wektorze.

Kompilatory C++ są na tyle inteligentne, że rozpoznają funkcję, o którą nam chodzi przez:

- nazwę funkcji (*add*),
- typy przyjmowanych argumentów (*TSubstance* i *double* **LUB** *string* i *double*).

Dzięki temu możemy mieć kilka funkcji o takiej samej nazwie (co sugeruje, że służą do tego samego), ale z różnymi parametrami wejściowymi (w zależności czym, tj. jakimi informacjami, dysponuje użytkownik).

Dodanie płynu po nazwie

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1   #pragma once
2   #include <iostream>
3   #include <vector>
4
5   #include "TSubstance.h"
6
7
8   class TCup
9   {
10      std::vector<TSubstance> substances;
11      std::vector<double> volumes;
12
13  public:
14      void add(TSubstance substance, double volume_in_ml);
15      void add(std::string name, double volume_in_ml);
16      void show();
17
18  private:
19      int get_substance_id(std::string name);
20  };
```

Linia 19

Tworzymy funkcję pomocniczą (czyli prywatną, do wewnętrznego użytku w klasie) służącą do znajdowania pozycji w wektorze cieczy o interesującej nas nazwie.

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | TCup | show()

```
4   void TCup::add(TSubstance substance, double volume_in_ml)
5   {
6       substances.push_back(substance);
7       volumes.push_back(volume_in_ml / 1e6);
8   }
9
10  void TCup::add(std::string name, double volume_in_ml)
11  {
12      int _id = get_substance_id(name);
13
14      if (_id >= 0)
15      {
16          this->add(substancje[_id], volume_in_ml);
17      }
18  }
```

Linia 12

Znajdź pozycję cieczy w wektorze po jej nazwie – póki co nie ważne jak to się odbywa.

Linia 14-17

Jeśli pozycja jest większa/równa 0 (odnaleziono ciecz) dodaj ją do kubka – korzystamy z wcześniejszej wersji funkcji *add*.

```
20 int TCup::get_substance_id(string name)
21 {
22     int count = substancje.size();
23     int _id = -1;
24     bool run = true;
25
26     while (run)
27     {
28         _id += 1;
29         string sub_name = substancje[_id].get_name();
30
31         if (sub_name == name) run = false;
32
33         if ((_id + 1 == count) and (run == true))
34         {
35             cout << "Nie znaleziono plynu o podanej nazwie: \"" << name << "\"!\n";
36             run = false;
37             _id = -1;
38         }
39     }
40
41     return _id;
42 }
```

Linia 22-24

Warunki początkowe – odczytujemy wielkość wektora, ustawiamy pozycje na -1.

Linia 28-29

Przechodzimy do nowej pozycji w wektorze (+1) i odczytujemy nazwę cieczy.

Linia 31

Jeśli nazwa cieczy się zgadza zatrzymujemy poszukiwania (pętle while).

Linia 33

Jeśli doszliśmy do końca wektora (_id+1 == count) i wcześniejszy warunek nie zadziałał (nie znaleźliśmy cieczy o podanej nazwie) wyrzucamy informacje o braku dopasowania (_id=-1) i kończymy poszukiwania (pętle while).

Linia 41

Zwracamy pozycję cieczy w wektorze. Pozycja -1 informuje, że ciecz nie została znaleziona.

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | TCup | show()

```cpp
void TCup::add(TSubstance substance, double volume_in_ml)
{
    substances.push_back(substance);
    volumes.push_back(volume_in_ml / 1e6);
}

void TCup::add(std::string name, double volume_in_ml)
{
    int _id = get_substance_id(name);

    if (_id >= 0)
    {
        this->add(substancje[_id], volume_in_ml);
    }
}

int TCup::get_substance_id(string name)
{
    int count = substancje.size();
    int _id = -1;
    bool run = true;

    while (run)
    {
        _id += 1;
        string sub_name = substancje[_id].get_name();

        if (sub_name == name) run = false;

        if ((_id + 1 == count) and (run == true))
        {
            cout << "Nie znaleziono plynu o podanej nazwie: \"" << name << "\"!\n";
            run = false;
            _id = -1;
        }
    }

    return _id;
}
```

```cpp
int main()
{
    cout << "Hello World!\n";

    TCup kubek1;
    kubek1.add(substancje[0], 100);
    kubek1.add(substancje[1], 50);
    kubek1.add("woda", 30);
    kubek1.add("kwas", 1);
    kubek1.show();
}
```

Konsola debugowania programu Microsoft Visual Studio

```
Hello World!
Nie znaleziono plynu o podanej nazwie: "kwas"!
0. woda; volume: 100 ml; mass: 99.78 g
1. mleko; volume: 50 ml; mass: 51.5 g
2. woda; volume: 30 ml; mass: 29.934 g
Volume coe: 55.5556 - 27.7778 - 16.6667 -
Mass coe: 55.062 - 28.4194 - 16.5186 -
```